
Bericht zum Praxismodul

Nils Tuchscheerer

Unternehmen: Fraunhofer-Institut für Techno-
und Wirtschaftsmathematik ITWM
Betreuer: Dr. Jan Mohring und Marvin Schulte
Zeitraum: 01. April bis 30. Juni 2024

Universität Kassel
Fachbereich 10 - Mathematik und Naturwissenschaften
Institut für Mathematik

Inhaltsverzeichnis

1	Einleitung	3
2	Vor der Praxisphase	3
3	Des Unternehmen	3
4	Der Praxisaufenthalt	4
4.1	Direkter Ansatz	4
4.1.1	Patankar-Verfahren	5
4.2	Optimale-Steuerung's Ansatz	7
4.2.1	Pontryagin's Maximums Prinzip und Forward-Backward-Sweep	8
4.2.2	Finite-Differenzen-Ansatz	10
4.2.3	Korrektur des FBS-Verfahrens	13
4.2.4	Vergleich der Verfahren	16
5	Zusammenfassung und Bewertung	19

1 Einleitung

Im Mathematikstudium hört man nicht selten, dass Mathematiker überall gesucht werden, doch bei vielen der typischen Berufe für Mathematiker geht es vor allem um die erlangte Denkweise und nicht direkt um Mathematik. In meinem Praktikum wollte ich mit Mathematik arbeiten, was ich vom 01.04.2024 bis zum 30.06.2024 am Fraunhofer ITWM in Kaiserslautern mit einem Fokus auf Numerik getan habe. Ich untersuchte Ansätze zur Implementierung einer simplen Pandemiemodellierung mit Patankar-Verfahren und erlangte so Einblicke in die Arbeitswelt und die Forschung.

Dabei habe ich viel über das praktische Arbeiten mit numerischen Verfahren gelernt und Einblicke in die Theorie mir neuer Teilgebiete der Numerik erhalten.

In diesem Bericht werde ich zunächst meinen Weg zum ITWM darlegen, danach folgen einige Informationen zur Fraunhofer-Gesellschaft und dem Fraunhofer ITWM und schließlich fasse ich meine Tätigkeiten während des Praktikums zusammen.

2 Vor der Praxisphase

Auf das Fraunhofer Institut für Techno- und Wirtschaftsmathematik (ITWM) wurde ich durch Kommilitonen und ein Gespräch mit Professor Meister aufmerksam. Über Professor Meister bin ich mit Dr. Hietel, dem Leiter der Abteilung Transportvorgänge (TV) am ITWM, in Kontakt gekommen und nachdem klar war, dass Interesse an einem Praktikum besteht, habe ich mich über das Bewerbungsportal der Fraunhofer-Gesellschaft beworben. Nach einem Bewerbungsgespräch per Teams Mitte Januar, hatte ich Anfang März meinen Vertrag. Zusätzlich habe ich vom „Haus der Mathematik“, einer Unterkunft, gedacht für Mathematikstudenten der Rheinland-Pfälzischen Technischen Universität Kaiserslautern-Landau, erfahren, in dem ich günstig untergekommen bin.

3 Des Unternehmen

Das Fraunhofer ITWM ist ein Institut der Fraunhofer-Gesellschaft, einer weltweit führenden Einrichtung für angewandte Forschung mit Sitz in Deutschland. Die Fraunhofer-Gesellschaft wurde 1949 gegründet und unterhält momentan 76 Standorte, mit verschiedenen Forschungsschwerpunkten, auf die sich insgesamt über Dreißig-tausend Mitarbeiter verteilen.

Im ITWM, welches am 9. November 1995 gegründet und zum 1. Januar 2001 in die Fraunhofer-Gesellschaft aufgenommen wurde, wird in 9 Abteilungen mit einem Fokus auf angewandte Mathematik geforscht. Ich war in der Abteilung Transportvorgänge und der Gruppe Energienetze und Modellreduktion, welche von Dr. Jan Mohring geleitet wird.

4 Der Praxisaufenthalt

Mit meinen Betreuern habe ich vor Beginn des Praktikums besprochen, dass ich mich vorbereitend mit Patankar-Runge-Kutta-Verfahren beschäftigen werde, was ich im Rahmen der Vorlesung „Numerik gewöhnlicher Differentialgleichungen“ tat. Das Besondere an diesem numerischen Lösungsverfahren für gewöhnliche Differentialgleichungen ist, dass theoretisch positive Größen auch numerisch positiv gehalten werden. Aufbauend darauf sollte ich im Rahmen des Praktikums untersuchen, wie sich diese Verfahren auf Genauigkeit und Stabilität eines simplen Coronapandemie-Modells auswirken können.

Zu Beginn des Praktikums wurde ich in Python und einige unterstützende Bibliotheken eingearbeitet. Ich habe mit „matplotlib“, benutzt zum grafischen Darstellen, „numpy“, welches Matlab-ähnliches Arbeiten mit Matrizen und Vektoren ermöglicht, und „scipy“, verwendet für weitere mathematische Methoden, gearbeitet. Durch meine Erfahrungen mit Java und Matlab aus dem Studium kam ich mit allem gut zurecht. Als nächsten Schritt implementierte ich ein einfaches Modell gedämpfter Pendelbewegung [1], zunächst mit einem ODE-Solver von scipy und anschließend mit einer eigenen Implementierung des Euler- [7] und des klassischen Runge-Kutta- [10] Verfahrens.

4.1 Direkter Ansatz

Der erste Ansatz zum Einbauen eines Patankar-Verfahrens war das direkte Implementieren eines Pandemie Modells, mit nicht negativen Größen, mit einem Patankar-Verfahren. Das Differentialgleichungsmodell, mit dem ich arbeitete, berechnet zu einem gegebenen Tag t die vor Infektion Geschützten $p(t)$ und die an diesem Tag Neuinfizierten $\nu(t)$ als

$$\begin{aligned}\nu'(t) &= \frac{1}{\tau} \left(1 - \frac{1}{r \cdot (1 - p(t))} \right) \nu(t) \\ p'(t) &= \nu(t) - \alpha \cdot p(t).\end{aligned}\tag{1}$$

Hierbei werden die Konstanten $\tau = 3,58$, der Mittelpunkt der infektiösen Phase, $\alpha = 0,03$, die Verlustrate des Infektionsschutzes unter den Geschützten und $r = 4$, die, zunächst konstant gewählte, Reproduktionsrate des Virus, verwendet. Weitere Informationen zu diesem Modell finden sich in [6].

Bevor ich mich den Patankar-Verfahren zuwandte, passte ich das Modell an, um die Vermeidung negativer Werte besser auszunutzen. Statt den Geschützten, die in einer Implementierung mit dem Runge-Kutta-Verfahren stets positiv waren, jedoch teilweise über 100% erreichten, modellierte ich die Anfälligen $s(t) = 1 - p(t)$. Für s stellte Positivität nun also eine Einschränkung dar, die praktisch zum Tragen kam. So ergab sich das System

$$\begin{aligned}\nu'(t) &= \frac{1}{\tau} \left(1 - \frac{1}{r \cdot s(t)} \right) \nu(t) \\ s'(t) &= \alpha \cdot (1 - s(t)) - \nu(t).\end{aligned}\tag{2}$$

4.1.1 Patankar-Verfahren

Die Patankar- oder Patankar-Runge-Kutta-Verfahren basieren auf einer Zerlegung der Ableitungsterme einer Differentialgleichung $x'(t) = f(x(t)) \in \mathbb{R}^n$ in Produktionsterme $P(x) \geq 0$ und Destruktionsterme $D(x) \geq 0$, so dass $f(x) = P(x) - D(x)$, $\forall x \in \mathbb{R}^n$ gilt. Sowohl die Produktions- als auch die Destruktionsterme $K(x)$ werden weiter aufgeteilt in Terme $K_i(x)$, die zur Ableitung von x_i beitragen, und $k_{i,j}(x)$, die zur Ableitung von x_i beitragen und von x_j abhängen, wobei gilt $x = (x_1, \dots, x_n)^T$, $i, j = 1, \dots, n$. Gearbeitet habe ich mit dem Patankar-Euler-Verfahren,

$$x_i^{n+1} = x_i^n + \Delta t \left(P_i(x^n) - D_i(x^n) \frac{x_i^{n+1}}{x_i^n} \right), \quad (3)$$

einem Verfahren erster Ordnung, und dem modifizierten Patankar-Runge-Kutta-Verfahren,

$$\begin{aligned} x_i^{(1)} &= x_i^n + \Delta t \left(\sum_{j=1}^n p_{i,j}(x^n) \frac{x_j^{(1)}}{x_j^n} - d_{i,j}(x^n) \frac{x_i^{(1)}}{x_i^n} \right) \\ x_i^{n+1} &= x_i^n + \frac{\Delta t}{2} \left(\sum_{j=1}^n (p_{i,j}(x^n) + p_{i,j}(x^{(1)})) \frac{x_j^{n+1}}{x_j^{(1)}} \right. \\ &\quad \left. - \sum_{j=1}^n (d_{i,j}(x^n) + d_{i,j}(x^{(1)})) \frac{x_i^{n+1}}{x_i^{(1)}} \right), \end{aligned} \quad (4)$$

welches von zweiter Ordnung ist. Weitere Informationen zu diesen Verfahren finden sich in [3].

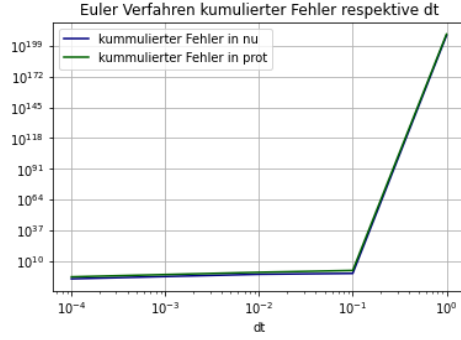
Das aus (2), $\nu(0) = 0.003$ und $s(0) = 1$ bestehende Anfangswertproblem implementierte ich zuerst mit dem Euler- und dem Patankar-Euler-Verfahren. Ich verglich beide Verfahren bezüglich des zu einer gegebenen Zeitschrittweite Δt auftretenden Fehlers.

Um diesen Vergleich durchzuführen, erstellte ich eine Referenzlösung mit dem Runge-Kutta Verfahren, mit welcher ich den kumulierten Fehler in den ersten 1000 Tagen bei gegebenem $\Delta t = 10^{-i}$ mit $i = 0, \dots, 4$ als Differenz des Ergebnisses des jeweiligen Verfahrens bei dieser Schrittweite zur Referenz-Lösung für alle $t \in \mathbb{N}_0 \cap [0, 1000]$ ermittelte.

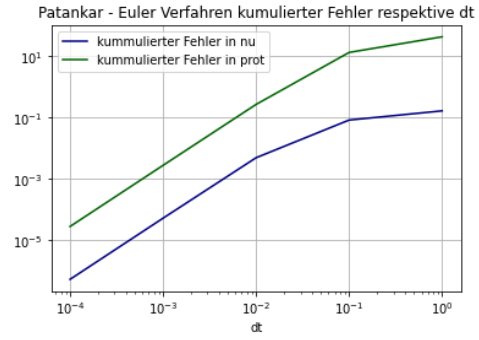
Abbildungen 1 und 2 ließen vermuten, dass das Patankar-Euler Verfahren verglichen zum Euler Verfahren etwas genauere Ergebnisse liefert und für größere Zeitschritte stabil ist.

Anschließend implementierte ich das Anfangswertproblem mit dem Runge-Kutta- und dem modifizierten Patankar-Runge-Kutta-Verfahren und führte den gleichen Vergleich durch.

Abbildungen 3 und 4 zeigen diesen Vergleich und lassen vermuten, dass das Runge-Kutta Verfahren schneller konvergiert und, dass das modifizierte Patankar-Runge-Kutta-Verfahren für größere Zeitschrittweiten stabil ist.

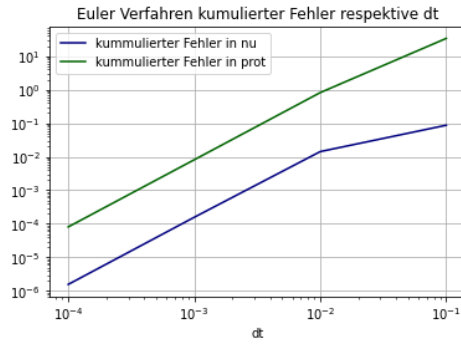


(a) Euler Verfahren

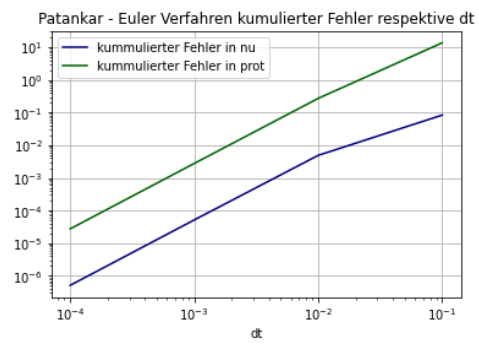


(b) Patankar-Euler Verfahren

Abbildung 1: Fehler in den ersten 1000 Tagen bei Schrittweite $\Delta t = dt$

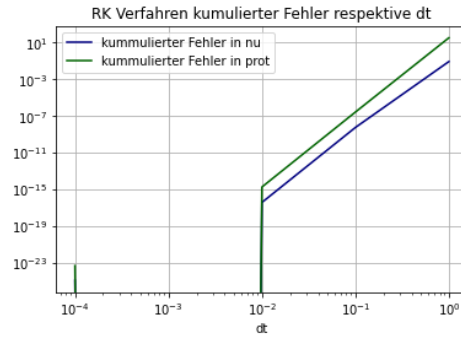


(a) Euler Verfahren

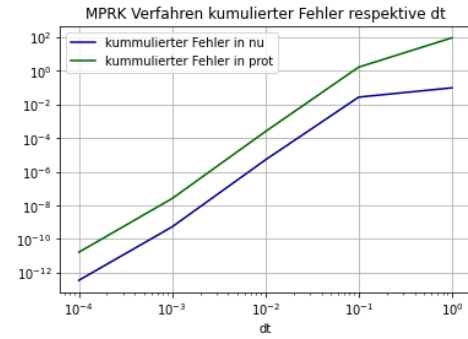


(b) Patankar Euler Verfahren

Abbildung 2: Fehler in den ersten 1000 Tagen bei Schrittweite $\Delta t = dt < 1$



(a) Runge-Kutta Verfahren



(b) Modifiziertes Patankar-Runge-Kutta-Verfahren

Abbildung 3: Fehler in den ersten 1000 Tagen bei Schrittweite $\Delta t = dt$

Insgesamt wurde befunden, dass an dieser Stelle das Runge-Kutta Verfahren bevorzugt werden sollte, da das modifizierte Patankar-Runge-Kutta-Verfahren eine akzeptable

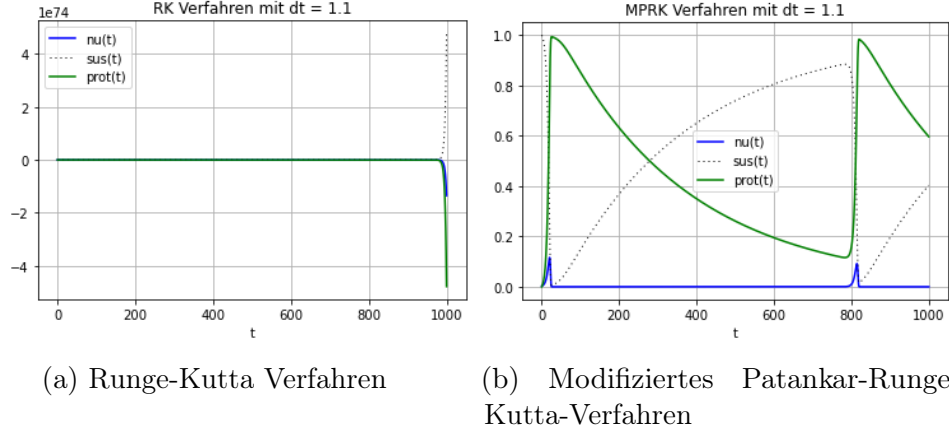


Abbildung 4: Ergebnis bei Schrittweite $\Delta t = dt = 1, 1$

Genauigkeit nicht schneller erreichte.

Hier sei noch angemerkt, dass der überraschend geringe Fehler des Runge-Kutta-Verfahrens bei $\Delta t = 10^{-3}$ darauf zurückzuführen ist, dass das hier berechnete Ergebnis die von mir verwendete Referenzlösung war.

4.2 Optimale-Steuerung's Ansatz

Eine andere Möglichkeit, Patankar-Typ-Verfahren einzusetzen, war das Berechnen der Reproduktionsrate $r(t)$, beziehungsweise der Kontaktrate $\kappa(t)$, wobei $r(t) = \delta\kappa(t)$, mit $\delta = 4$ als durchschnittliche Länge der infektiösen Phase, gilt. Durch das Verwenden der Kontaktrate wurde die Differentialgleichung zu

$$\begin{aligned}\nu'(t) &= \frac{1}{\tau} \left(1 - \frac{1}{\delta\kappa(t)s(t)} \right) \nu(t) \\ s'(t) &= \alpha \cdot (1 - s(t)) - \nu(t).\end{aligned}\tag{5}$$

Ziel war es nun, eine Kontaktrate zu berechnen, für welche die berechnete Zahl der Infektiösen $i(t) = \delta(\nu(t) - \tau\nu'(t))$ (nach [6]) möglichst geringfügig von Messdaten abweicht, ohne dass diese Kontaktrate unrealistisch stark schwankt. Die zu diesem Zweck von mir verwendeten Daten $\pi(t)$ sind eine Interpolation der Messwerte der SentiSurv-Erhebung der Universitätsmedizin Mainz bis zum 18.10.2023, siehe [6]. Aus diesen Daten habe ich die gemessenen Infektiösen als die Zahl der positiven Testergebnisse auf die Gesamtzahl der Getesteten berechnet, wie in Abbildung 5a zu sehen. Hier sei angemerkt, dass zu Beginn der Erhebung die Gesamtzahl der Getesteten recht klein war, wie in Abbildung 5b zu sehen. Entsprechend ist die Datengrundlage zu Beginn des Zeitintervalls weniger aussagekräftig.

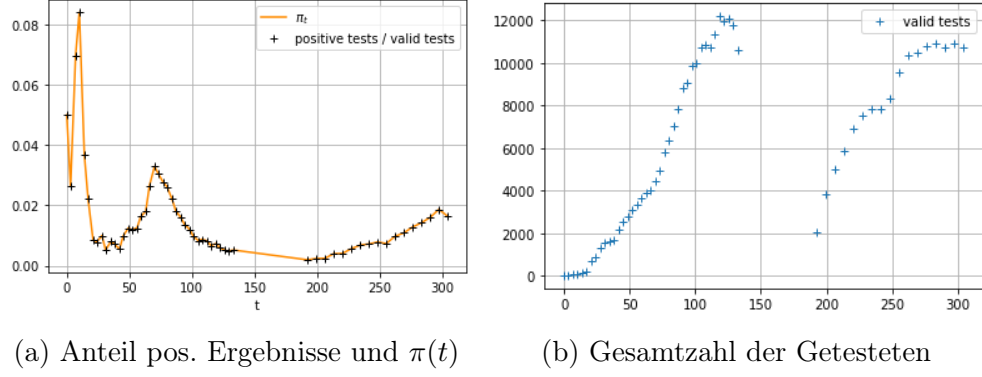


Abbildung 5: Datengrundlage

Es wurde also ein $\kappa(t)$ gesucht, das die Funktion

$$J\left(\begin{pmatrix} \nu(t) \\ s(t) \end{pmatrix}, \kappa(t)\right) = \int_{t_0}^{t_{end}} -A(i(t) - \pi(t))^2 - B(\kappa'(t))^2 dt \quad (6)$$

maximiert, wobei A und B die Gewichte der Zielfunktion sind. Um ein geeignetes Verfahren zu finden, befasste ich mich mit der Theorie der optimalen Steuerung.

4.2.1 Pontryagin's Maximums Prinzip und Forward-Backward-Sweep

Von mir wurde ein Verfahren auf Basis des Pontryagin'schen Maximums Prinzips [2] verwendet. Dieses betrachtet eine zu maximierende Gewinnfunktion

$$J(x(t), u(t)) = \int_{t_0}^{t_{end}} r(x(t), u(t)) dt + g(x(t_{end})),$$

abhängig von einer Funktion $x(t)$ und einer Steuerung $u(t)$. Hinzu kommt die Forderung, dass $x(t)$ eine Lösung des Anfangswertproblems $x(t_0) = x_0$, $x'(t) = f(t, x(t), u(t))$ ist. Um J zu maximieren, werden eine adjungierte Funktion $y(t)$, welche die Rolle eines Lagrange-Multiplikators hat, und eine Hamilton-Funktion $H(t) = f \cdot y + r$ hinzugezogen. Die adjungierte fungiert nur als Zwischenwert, wohingegen die Hamilton-Funktion genutzt wird, da die Steuerung, welche H maximiert, auch J unter der Einschränkung des Anfangswertproblems maximiert. Die adjungierte wird über das umgekehrte Anfangswertproblem $y(t_{end}) = -\nabla g(x(t_{end}))$, $y'(t) = -\nabla_x H$ definiert. Im simpelsten Fall kann über $\frac{\partial}{\partial u} H = 0$ direkt die optimale Steuerung berechnet werden.

Die Forward-Backward-Sweep Methode (FBS) [4] wird mit einer Steuerung $u_0(t)$ initialisiert und implementiert diese Theorie in drei Iterationsschritten.

- Forward: Vorwärts-Lösen des Anfangswertproblems zur Berechnung von $x_n(t)$

- Backward: Rückwärts-Lösen des umgekehrten Anfangswertproblems zur Berechnung von $y_n(t)$
- Sweep: Berechnung der neuen Steuerung $u_n(t)$.

Zum Sweep-Schritt sei angemerkt, dass u_n oft nicht direkt als neue Steuerung übernommen wird, sondern zum Beispiel $\frac{u_n - u_{n-1}}{2}$ als Steuerung genutzt wird, was zu schnellerer Konvergenz führen soll.

Um wie gewollt $\kappa'(t)$ bestrafen zu können, habe ich die Differentialgleichung um $\kappa'(t) = u(t)$ ergänzt, wobei $u(t)$ die Steuerung des Systems war.

Das FBS-Verfahren implementierte ich zunächst mit dem Runge-Kutta-Verfahren zum Lösen der Anfangswertprobleme und dem Lösen einer linearen Gleichung zum Berechnen von u . Als Konvergenzkriterium wählte ich zunächst $\sup_{t \in [t_0, t_{end}]} \{u_n(t) - u_{n-1}(t)\} > tol$ mit einer hinreichend kleinen Toleranz tol .

Mit der Gewinnfunktion (6) gestaltete sich das Verfahren zunächst als schwer zu implementieren, weshalb ich die Gewinnfunktion vorerst zu

$$J\left(\begin{pmatrix} \nu(t) \\ s(t) \end{pmatrix}, \kappa(t)\right) = \int_{t_0}^{t_{end}} -A(\nu(t) - \nu_s(t))^2 - Bu(t)^2 dt \quad (7)$$

vereinfachte. Hierbei war $\nu_s(t)$ eine synthetische Lösung für $\nu(t)$, welche per Vorwärtssimulation mit einer synthetischen, willkürlich gewählten Kontaktrate berechnet wurde und als Ersatz für einen Messwert diente.

Insgesamt ergab sich das System

$$\begin{aligned} J\left(\begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t)\right) &= \int_{t_0}^{t_{end}} -A(\nu(t) - \nu_s(t))^2 - Bu(t)^2 dt \\ H\left(t, \begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t), \begin{pmatrix} y_1(t) \\ y_2(t) \\ y_3(t) \end{pmatrix}\right) &= \nu'(t)y_1(t) + s'(t)y_2(t) + \kappa'(t)y_3(t) \\ &\quad - A(\nu(t) - \nu_s(t))^2 - Bu(t)^2 \\ \nu(t_0) &= \nu_0 \\ \nu'(t) &= \frac{1}{\tau} \left(1 - \frac{1}{\delta \kappa(t)s(t)}\right) \nu(t) \\ s(t_0) &= s_0 \\ s'(t) &= \alpha \cdot (1 - s(t)) - \nu(t) \\ \kappa(t_0) &= \kappa_0 \\ \kappa'(t) &= u(t) \\ y_1(t_{end}) &= 0 \end{aligned} \quad (8)$$

$$\begin{aligned}
y_1'(t) &= -\frac{1}{\tau} \left(1 - \frac{1}{\delta\kappa(t)s(t)} \right) y_1(t) \\
&\quad + 2A(\nu(t) - \nu_s(t)) \\
y_2(t_{end}) &= 0 \\
y_2'(t) &= \alpha y_2(t) - \frac{\nu(t)y_1(t)}{\tau\delta\kappa(t)s^2(t)} \\
y_3(t_{end}) &= 0 \\
y_3'(t) &= -\frac{\nu(t)y_1(t)}{\tau\delta\kappa^2(t)s(t)} \\
\frac{\partial}{\partial u} H &= y_3(t) - 2Bu(t) = 0,
\end{aligned}$$

wobei $y_i(t)$, $i = 1, 2, 3$ die Adjungierten waren.

Meine Implementierung dieses Systems war bei synthetischen Neuinfizierten $\nu_s(t)$, die mit einer nicht-konstanten synthetischen Steuerung berechnet wurden, für Zeitintervalle von über 250 Tagen sehr instabil und kleine Änderungen an Anfangswerten oder Gewichten ließen das Verfahren auf undurchsichtige Weise zwischen Konvergenz und Divergenz schwanken. Angesichts dieser Instabilitäten wurde vermutet, dass FBS in dieser Form zum Lösen von (8) ungeeignet war.

4.2.2 Finite-Differenzen-Ansatz

Als nächstes versuchte ich, (8) mit einem Finite-Differenzen-Ansatz [8] zu lösen. Finite-Differenzen-Methoden sind eine Klasse numerischer Verfahren zum Lösen gewöhnlicher und partieller Differentialgleichungen, die auf Diskretisierung der veränderlichen, meist der Zeit, basieren. Statt dem Zeitintervall $[t_0, t_{end}]$ betrachtet man ein, meist äquidistantes, Zeitgitter $\{t_0, t_1, \dots, t_{end}\}$. Die Voraussetzung der Differentialgleichung an die Ableitung der Funktion in jedem Zeitpunkt t_i wird als Nullstellensuche formuliert, welche mit den gängigen Methoden gelöst werden kann. Um die Differentialgleichung als Nullstellensuche zu formulieren, werden Differenzenquotienten genutzt, um die Ableitung zu approximieren. Beispielsweise wird aus dem Anfangswertproblem

$$x(t_0) = x_0, x'(t) = f(x(t)), t \in [t_0, t_{end}]$$

mit der Diskretisierung $\{t_0, t_1 = t_0 + 1 \cdot \Delta t, \dots, t_n = t_0 + n \cdot \Delta t = t_{end}\}$, $\Delta t > 0$ und dem Differenzenquotient $\frac{x(t_i) - x(t_{i-1})}{\Delta t} \approx x'(t_i)$ die Funktion

$$g_{\Delta t}(x) := g(x) = g \left(\begin{pmatrix} x(t_0) \\ x(t_1) \\ \vdots \\ x(t_n) \end{pmatrix} \right) = \begin{pmatrix} x(t_0) - x_0 \\ \frac{x(t_1) - x(t_0)}{\Delta t} - f(x(t_1)) \\ \vdots \\ \frac{x(t_n) - x(t_{n-1})}{\Delta t} - f(x(t_n)) \end{pmatrix},$$

welche als Nullstelle die Funktion $\hat{x}_{\Delta t}$ hat. Da der Differenzenquotient so gewählt wurde, dass er für $\Delta t \rightarrow 0$ gegen die Ableitung von x konvergiert, liefert $\hat{x}_{\Delta t}$ für kleiner werdende $\Delta t > 0$ immer bessere Näherungslösungen an das Anfangswertproblem.

Um nun die Nullstelle einer solchen Funktion g zu finden, gibt es einige bekannte Verfahren, von denen ich das Newtonverfahren [11] wählte.

Die Iterationsformel lautet also

$$x^{n+1} = x^n + \Delta x^n, \quad Dg(x^n)\Delta x^n = -g(x^n),$$

wobei $Dg(x)$ die Jacobimatrix von $g(x)$ meint.

Um (8) zu modellieren, verwendete ich das Schema $\frac{x(t_{i+1})-x(t_{i-1}))}{2\Delta t} - x'(t_i)$ für $t_i \in \{t_1, \dots, t_{n-1}\}$, $x \in \{\nu, s, \kappa, y_1, y_2, y_3\}$. Für die Randpunkte verwendete ich die Schemata $\frac{3x(t_0)-4x(t_1)+x(t_2))}{2\Delta t} - x'(t_0)$ für den ersten Gitterpunkt t_0 und $x \in \{y_1, y_2, y_3\}$ und $\frac{3x(t_n)-4x(t_{n-1})+x(t_{n-2}))}{2\Delta t} - x'(t_n)$ für den letzten Gitterpunkt t_n und $x \in \{\nu, s, \kappa\}$, wobei die jeweils linke Seite der Ableitung einer, durch die drei einfließenden Punkte interpolierten, Parabel, entspricht. Für u verwendete ich den Term $u(t_i) - 2By_3(t_i)$ für $t_i \in \{t_0, \dots, t_n\}$, welcher direkt (8) entspringt.

Insgesamt ergab sich die Iteration

$$Df \begin{pmatrix} \nu^n(t) \\ s^n(t) \\ \kappa^n(t) \\ y_1^n(t) \\ y_2^n(t) \\ y_3^n(t) \\ u^n(t) \end{pmatrix} \cdot \begin{pmatrix} \Delta \nu^n(t) \\ \Delta s^n(t) \\ \Delta \kappa^n(t) \\ \Delta y_1^n(t) \\ \Delta y_2^n(t) \\ \Delta y_3^n(t) \\ \Delta u^n(t) \end{pmatrix} = f \begin{pmatrix} \nu^n(t) \\ s^n(t) \\ \kappa^n(t) \\ y_1^n(t) \\ y_2^n(t) \\ y_3^n(t) \\ u^n(t) \end{pmatrix} \quad (9)$$

$$\begin{pmatrix} \nu^{n+1}(t) \\ s^{n+1}(t) \\ \kappa^{n+1}(t) \\ y_1^{n+1}(t) \\ y_2^{n+1}(t) \\ y_3^{n+1}(t) \\ u^{n+1}(t) \end{pmatrix} = \begin{pmatrix} \nu^n(t) \\ s^n(t) \\ \kappa^n(t) \\ y_1^n(t) \\ y_2^n(t) \\ y_3^n(t) \\ u^n(t) \end{pmatrix} + \begin{pmatrix} \Delta \nu^n(t) \\ \Delta s^n(t) \\ \Delta \kappa^n(t) \\ \Delta y_1^n(t) \\ \Delta y_2^n(t) \\ \Delta y_3^n(t) \\ \Delta u^n(t) \end{pmatrix},$$

mit

$$f \begin{pmatrix} \begin{pmatrix} \nu(t_0) \\ \nu(t_1) \\ \vdots \\ \nu(t_{n-1}) \\ \nu(t_n) \\ s(t_0) \\ s(t_1) \\ \vdots \\ s(t_{n-1}) \\ s(t_n) \\ \kappa(t_0) \\ \kappa(t_1) \\ \vdots \\ \kappa(t_{n-1}) \\ \kappa(t_n) \\ y_1(t_0) \\ y_1(t_1) \\ \vdots \\ y_1(t_{n-1}) \\ y_1(t_n) \\ y_2(t_0) \\ y_2(t_1) \\ \vdots \\ y_2(t_{n-1}) \\ y_2(t_n) \\ y_3(t_0) \\ y_3(t_1) \\ \vdots \\ y_3(t_{n-1}) \\ y_3(t_n) \\ u(t_0) \\ \vdots \\ u(t_n) \end{pmatrix} \end{pmatrix} := \begin{pmatrix} \begin{matrix} \nu_0 \\ \frac{\nu(t_{1+1})-\nu(t_{1-1})}{2\Delta t} - \nu'(t_1) \\ \vdots \\ \frac{\nu(t_{n-1+1})-\nu(t_{n-1-1})}{2\Delta t} - \nu'(t_{n-1}) \\ \frac{3\nu(t_n)-4\nu(t_{n-1})+\nu(t_{n-2})}{2\Delta t} - \nu'(t_n) \end{matrix} \\ \begin{matrix} s_0 \\ \frac{s(t_{1+1})-s(t_{1-1})}{2\Delta t} - s'(t_1) \\ \vdots \\ \frac{s(t_{n-1+1})-s(t_{n-1-1})}{2\Delta t} - s'(t_{n-1}) \\ \frac{3s(t_n)-4s(t_{n-1})+s(t_{n-2})}{2\Delta t} - s'(t_n) \end{matrix} \\ \begin{matrix} \kappa_0 \\ \frac{\kappa(t_{1+1})-\kappa(t_{1-1})}{2\Delta t} - \kappa'(t_1) \\ \vdots \\ \frac{\kappa(t_{n-1+1})-\kappa(t_{n-1-1})}{2\Delta t} - \kappa'(t_{n-1}) \\ \frac{3\kappa(t_n)-4\kappa(t_{n-1})+\kappa(t_{n-2})}{2\Delta t} - \kappa'(t_n) \end{matrix} \\ \begin{matrix} \frac{3y_1(t_0)-4y_1(t_1)+y_1(t_2)}{2\Delta t} - y'_1(t_0) \\ \frac{y_1(t_{1+1})-y_1(t_{1-1})}{2\Delta t} - y'_1(t_1) \\ \vdots \\ \frac{y_1(t_{n-1+1})-y_1(t_{n-1-1})}{2\Delta t} - y'_1(t_{n-1}) \\ 0 \\ \frac{3y_2(t_0)-4y_2(t_1)+y_2(t_2)}{2\Delta t} - y'_2(t_0) \\ \frac{y_2(t_{1+1})-y_2(t_{1-1})}{2\Delta t} - y'_2(t_1) \\ \vdots \\ \frac{y_2(t_{n-1+1})-y_2(t_{n-1-1})}{2\Delta t} - y'_2(t_{n-1}) \\ 0 \\ \frac{3y_3(t_0)-4y_3(t_1)+y_3(t_2)}{2\Delta t} - y'_3(t_0) \\ \frac{y_3(t_{1+1})-y_3(t_{1-1})}{2\Delta t} - y'_3(t_1) \\ \vdots \\ \frac{y_3(t_{n-1+1})-y_3(t_{n-1-1})}{2\Delta t} - y'_3(t_{n-1}) \\ 0 \\ u(t_0) - 2By_3(t_0) \\ \vdots \\ u(t_n) - 2By_3(t_n) \end{matrix} \end{pmatrix}. \quad (10)$$

Initialisiert habe ich das Ganze mit einer Konstanten Steuerung $u(t) = 0$ und den damit mit dem Runge-Kutta-Verfahren berechneten Werten für $\nu(t)$, $s(t)$ und $\kappa(t)$. Um sicherzustellen, dass die partiellen Ableitungen in Df richtig berechnet wurden, habe ich sie mit sympy, einer Python-Bibliothek für symbolisches Rechnen, berechnet.

Diese Implementierung von (8) war wie das FBS-Verfahren sehr instabil für zu große Zeitspannen.

4.2.3 Korrektur des FBS-Verfahrens

Nach weiterer Recherche fand ich in [5] einen Fehler in meiner Implementierung des FBS-Verfahrens. Die Steuerung hat sich zu schnell verändert, was zu Divergenz des Verfahrens führen kann.

Um diesen Fehler zu beheben, passte ich meine Implementierung des FBS Verfahrens analog zur Armijo-Bedingung [9] an. Ich ergänzte eine Schrittweite h_n für u_n die in jedem Iterationsschritt n so gewählt wird, dass die zu maximierende Funktion J größer wird, was Divergieren des Verfahrens verhinderte. Für u_n ergab sich die Iteration

$$\begin{aligned}\Delta u_n &= \hat{u}_{n+1} - u_n, \\ u_{n+1} &= u_n + h_n \Delta u_n,\end{aligned}\tag{11}$$

wobei $\hat{u}_{n+1}$ die im Schritt $n+1$ durch das FBS Verfahren berechnete Steuerung bezeichnet. Um ein passendes h_n zu berechnen, führte ich die Parameter h_{init} und h_{div} , sowie die Abbruchbedingung $h_n < h_{min} = 10^{-13}$ ein. In jedem Schritt berechnete ich h_n als

$$h_n = \max_{k \in \mathbb{N}} \{h_n^k \mid J_{n+1} > J_n + \sigma h_n^k (J_{n+1} - J_n) \wedge h_n^k > h_{min}\},$$

wobei

$$\begin{aligned}h_n^1 &= h_{init}, \\ h_n^{k+1} &= \frac{h_n^k}{h_{div}}, \\ J_n &= J\left(\begin{pmatrix} \nu^n(t) \\ s^n(t) \\ \kappa^n(t) \end{pmatrix}, u^n(t)\right), \\ J_{n+1} &= J\left(\begin{pmatrix} \nu^{n+1}(t) \\ s^{n+1}(t) \\ \kappa^{n+1}(t) \end{pmatrix}, u^{n+1}(t)\right), \text{ mit } h_n = h_n^k,\end{aligned}\tag{12}$$

σ , bezeichnet die Armijo-Konstante.

Zunächst wählte ich die Werte $h_{init} = 10^{-2}$, $h_{div} = 2$ und $\sigma = 10^{-3}$.

Zuletzt änderte ich das Konvergenzkriterium im FBS-Verfahren zu $(J_n - J_{n-1}) > tol$. Mit diesen Anpassungen war das Verfahren ausreichend stabil. Je nach Wahl der Parameter galt meist $h_n \in [10^{-8}, 10^{-2}]$, was bewirkte, dass das Verfahren teilweise erst nach einigen hundert Iterationen fast zum Stillstand kam.

Nun war es auch möglich, das System mit (6), also insgesamt

$$J\left(\begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t)\right) = \int_{t_0}^{t_{end}} -A(i(t) - \pi(t))^2 - Bu(t)^2 dt$$

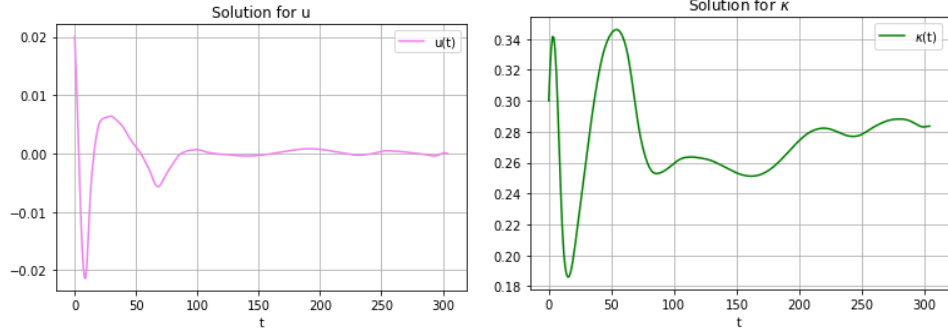
$$\begin{aligned}
H(t, \begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t), \begin{pmatrix} y_1(t) \\ y_2(t) \\ y_3(t) \end{pmatrix}) &= \nu'(t)y_1(t) + s'(t)y_2(t) + \kappa'(t)y_3(t) \\
&\quad - A(i(t) - \pi(t))^2 - Bu(t)^2 \\
\nu(t_0) &= \nu_0 \\
\nu'(t) &= \frac{1}{\tau} \left(1 - \frac{1}{\delta\kappa(t)s(t)} \right) \nu(t) \\
s(t_0) &= s_0 \\
s'(t) &= \alpha \cdot (1 - s(t)) - \nu(t) \\
\kappa(t_0) &= \kappa_0 \\
\kappa'(t) &= u(t) \\
y_1(t_{end}) &= 0 \\
y_1'(t) &= -\frac{1}{\tau} \left(1 - \frac{1}{\delta\kappa(t)s(t)} \right) y_1(t) \\
&\quad + 2A(i(t) - \pi(t)) \frac{1}{\kappa(t)s(t)} \\
y_2(t_{end}) &= 0 \\
y_2'(t) &= \alpha y_2(t) - \frac{\nu(t)y_1(t)}{\tau\delta\kappa(t)s^2(t)} \\
&\quad - 2A(i(t) - \pi(t)) \frac{\nu(t)}{\kappa(t)s^2(t)} \\
y_3(t_{end}) &= 0 \\
y_3'(t) &= -\frac{\nu(t)y_1(t)}{\tau\delta\kappa^2(t)s(t)} - 2A(i(t) - \pi(t)) \frac{\nu(t)}{\kappa^2(t)s(t)} \\
\frac{\partial}{\partial u} H &= y_3(t) - 2Bu(t) = 0,
\end{aligned} \tag{13}$$

zu implementieren. Hier sei angemerkt, dass $i(t) = \delta(\nu(t) - \tau\nu'(t)) = \frac{\nu(t)}{\kappa(t)s(t)}$.

Abbildung 6 zeigt das Ergebnis des Algorithmus mit $\nu(0) = 0,007$, $s(0) = 0,993$, $\kappa(0) = 0,3$, $A = 10$, $B = 1$, $\Delta t = 1$ und $tol = 10^{-10}$. Auffällig ist der vergleichsweise schlechte Fit zu Beginn, welcher jedoch in Anbetracht von Abbildung 5b, also aufgrund der geringen Verlässlichkeit dieser anfänglichen Daten, akzeptabel ist. Ansonsten werden die Datenpunkte gut getroffen und die Kontaktrate ist ausreichend glatt.

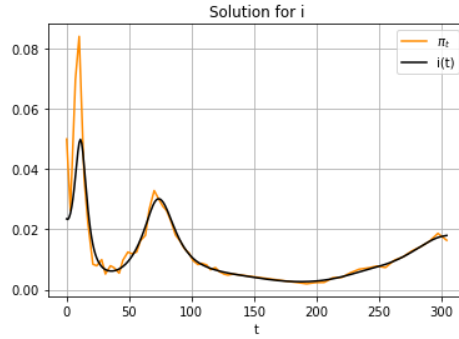
Als Nächstes habe ich das modifizierte Patankar-Runge-Kutta-Verfahren eingebunden, indem ich den Forward-Schritt des FBS-Verfahrens mit diesem implementiert habe, was vergleichbare Ergebnisse lieferte.

Anschließend habe ich die Steuerung $u(t)$ und die Adjungierten $y_1(t)$, $y_2(t)$, $y_3(t)$ nur auf $[t_0, t_u]$ berechnet und $u(t)$ für den Rest des Intervalls mit $u(t) = 0$ fortgesetzt, wie in Abbildung 7 mit $t_u = 270$ zu sehen, um eine Vorhersage zu erhalten.



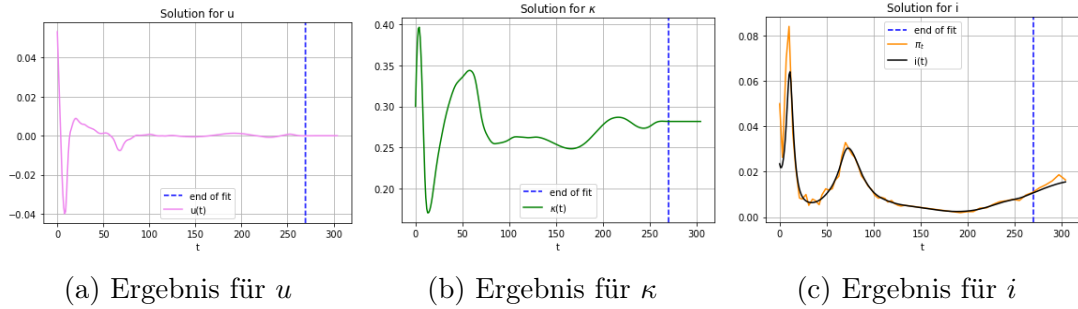
(a) Ergebnis für u

(b) Ergebnis für κ



(c) Ergebnis für i

Abbildung 6: Fit



(a) Ergebnis für u

(b) Ergebnis für κ

(c) Ergebnis für i

Abbildung 7: Vorhersage, Forward-Schritt mit dem Runge-Kutta-Verfahren

Zu diesem Zeitpunkt kam es vor, dass das Verfahren mit dem modifizierten Patankar-Runge-Kutta-Verfahren im Forward-Schritt für bestimmte Parameter nicht konvergierte, was auf eine Kontakttrate die zwischenzeitlich zu gering wurde zurückgeführt werden konnte. Um dieses Problem zu beheben wurde eine minimale Kontakttrate κ_{min} eingefügt, was zu dem System

$$J\left(\begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t)\right) = \int_{t_0}^{t_u} -A(i(t) - \pi(t))^2 - Bu(t)^2 dt$$

$$\begin{aligned}
H(t, \begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t), \begin{pmatrix} y_1(t) \\ y_2(t) \\ y_3(t) \end{pmatrix}) &= \nu'(t)y_1(t) + s'(t)y_2(t) + \kappa'(t)y_3(t) \\
&\quad - A(i(t) - \pi(t))^2 - Bu(t)^2, \quad t \in [t_0, t_u] \\
\nu(t_0) &= \nu_0 \\
\nu'(t) &= \frac{1}{\tau} \left(1 - \frac{1}{\delta(\kappa(t) + \kappa_{min})s(t)} \right) \nu(t) \\
s(t_0) &= s_0 \\
s'(t) &= \alpha \cdot (1 - s(t)) - \nu(t) \\
\kappa(t_0) &= \kappa_0 - \kappa_{min} \\
\kappa'(t) &= u(t) \\
y_1(t_u) &= 0 \\
y_1'(t) &= -\frac{1}{\tau} \left(1 - \frac{1}{\delta\kappa(t)s(t)} \right) y_1(t) \\
&\quad + 2A(i(t) - \pi(t)) \frac{1}{\kappa(t)s(t)} \\
y_2(t_u) &= 0 \\
y_2'(t) &= \alpha y_2(t) - \frac{\nu(t)y_1(t)}{\tau\delta\kappa(t)s^2(t)} \\
&\quad - 2A(i(t) - \pi(t)) \frac{\nu(t)}{\kappa(t)s^2(t)} \\
y_3(t_u) &= 0 \\
y_3'(t) &= -\frac{\nu(t)y_1(t)}{\tau\delta\kappa^2(t)s(t)} - 2A(i(t) - \pi(t)) \frac{\nu(t)}{\kappa^2(t)s(t)} \\
\frac{\partial}{\partial u} H &= y_3(t) - 2Bu(t) = 0,
\end{aligned} \tag{14}$$

mit $i(t) = \frac{\nu(t)}{(\kappa(t) + \kappa_{min})s(t)}$, führte. Für die meisten Fälle erwies sich $\kappa_{min} = 0,04$ als hinreichend groß um Konvergenz des Verfahrens zu gewährleisten und ausreichend klein, um die Ergebnisse nicht merklich zu beeinflussen.

4.2.4 Vergleich der Verfahren

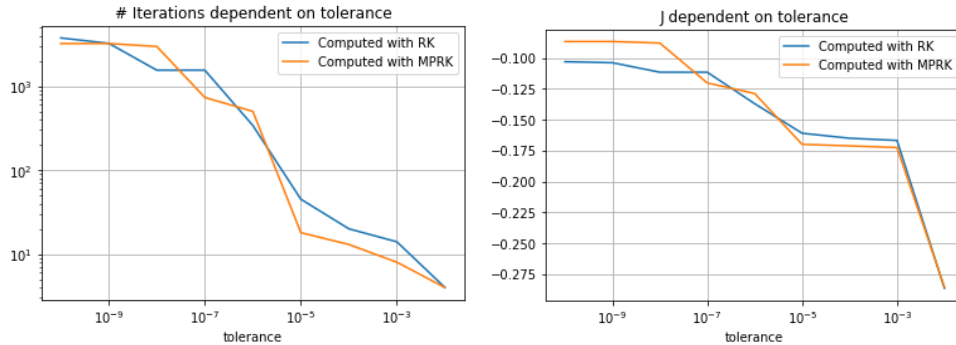
Da sowohl das Runge-Kutta- als auch das modifizierte Patankar-Runge-Kutta-Verfahren nun zufriedenstellend implementiert waren, galt es nun die beiden Verfahren zu vergleichen. Zunächst betrachtete ich die Zeitschrittweite Δt , die Toleranz im FBS-Verfahrens tol und den Schrittgrößenteiler in u , h_{div} als mögliche veränderliche. Die restlichen

Variablen setzte ich wie folgt:

$$\begin{aligned}
\nu(0) &= 0,007 \\
s(0) &= 0,993 \\
\kappa_{min} &= 0,04 \\
\kappa(0) &= 0,3 - \kappa_{min} \\
\sigma &= 10^{-3} \\
t_u &= 270 \\
h_{init} &= 10^{-2} \\
h_{min} &= 10^{-13} \\
\kappa_{min} &= 0,04 \\
A &= 10 \\
B &= 1
\end{aligned} \tag{15}$$

Ich verglich das Verhalten des FBS-Verfahrens bei Implementation des Forward-Schritts mit dem Runge-Kutta-Verfahren (RK) und mit dem modifizierten Patankar-Runge-Kutta-Verfahren (MPRK). In Abhängigkeit einer gewählten veränderlichen betrachtete ich die Zahl der Iterationen bis das FBS-Verfahren die gegebene Toleranz erreicht hatte und den Wert der Gewinnfunktion J , der erreicht wurde.

Die Abbildungen 8 und 9 zeigen, dass sich beide Varianten bei wachsender Toleranz wie zu Erwarten verhalten. Die Zahl der Iterationen und der erreichte Wert der Gewinnfunktion sinken. Es ist jedoch nicht zu erkennen, dass eine der Varianten allgemein schneller konvergiert oder ein besseres Ergebnis erzielt.

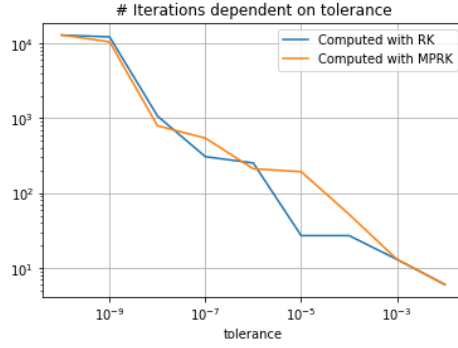


(a) Zahl Iterationen

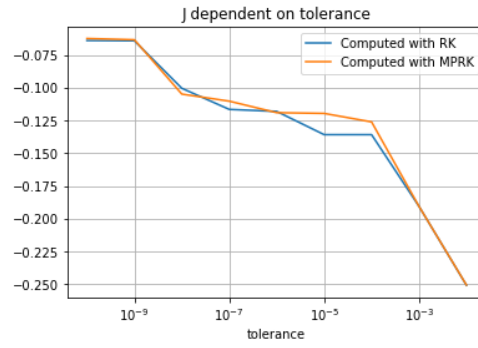
(b) Erreichter Wert der Gewinnfunktion

Abbildung 8: Vergleich mit $\Delta t = 4$, $tol \in \{10^{-10}, 10^{-9}, \dots, 10^{-2}\}$ und $h_{div} = 4$

Abbildung 10 zeigt das weniger eindeutige Verhalten bei unterschiedlichen Zeitschrittweiten. Hier erwartete ich bei größer werdendem Δt eine wachsende Zahl Iterationen und einen kleineren Wert der Gewinnfunktion, was jedoch im Allgemeinen nicht gilt.

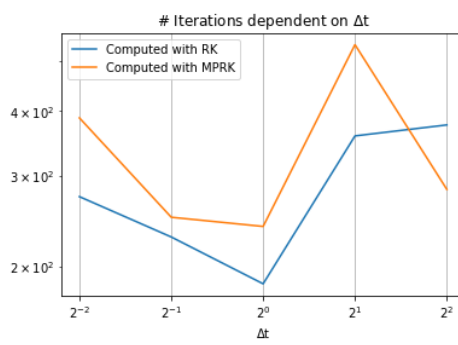


(a) Zahl Iterationen

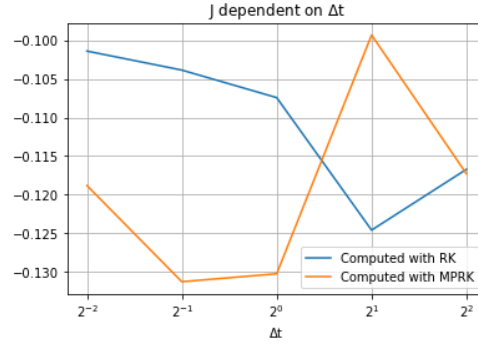


(b) Erreichter Wert der Gewinnfunktion

Abbildung 9: Vergleich mit $\Delta t = 1$, $tol \in \{10^{-10}, 10^{-9}, \dots, 10^{-2}\}$ und $h_{div} = 10$

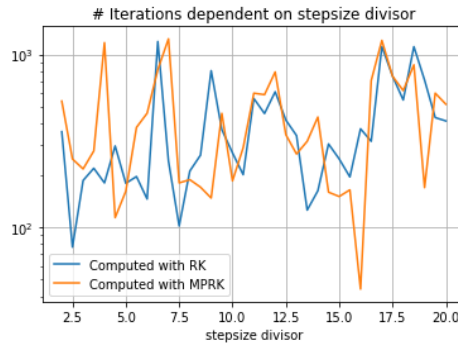


(a) Zahl Iterationen

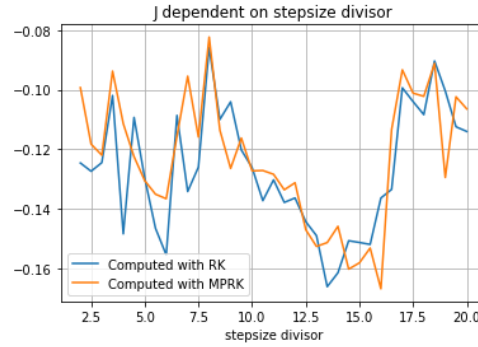


(b) Erreichter Wert der Gewinnfunktion

Abbildung 10: Vergleich mit $\Delta t \in \{2^{-2}, 2^{-1}, \dots, 2^2\}$, $tol = 10^{-6}$ und $h_{div} = 2$



(a) Zahl Iterationen



(b) Erreichter Wert der Gewinnfunktion

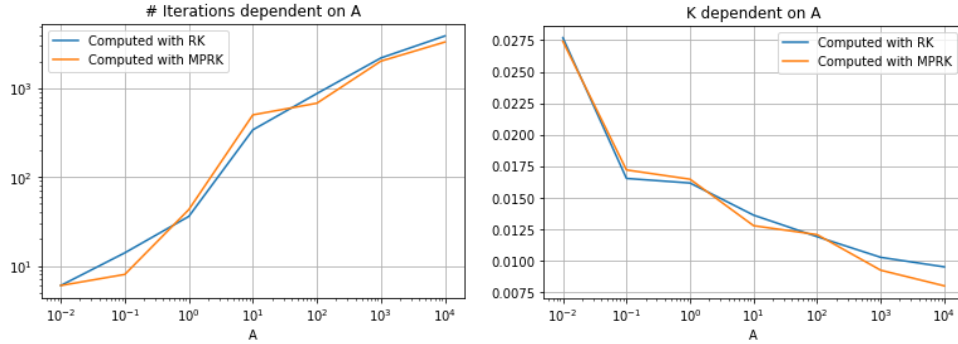
Abbildung 11: Vergleich mit $\Delta t = 2$, $tol = 10^{-6}$ und $h_{div} \in \{2, 0; 2, 5; \dots; 20\}$

Abbildung 11 zeigt das ausgesprochen chaotische Verhalten bei unterschiedlichen h_{div} -Werten. Hier hatte ich deutlich weniger Sprunghaftigkeit erwartet.

Zuletzt betrachtete ich das Verhalten der Varianten für verschiedene A -Werte. Hier betrachtete ich zum einen wieder die Zahl der Iterationen und zum anderen den Wert der Funktion K mit

$$K\left(\begin{pmatrix} \nu(t) \\ s(t) \\ \kappa(t) \end{pmatrix}, u(t)\right) = \int_{t_0}^{t_u} (i(t) - \pi(t))^2 dt, \quad (16)$$

um allein die Genauigkeit des Fits zu Bewerten. Hier ergab sich für beide Verfahren das zu erwartende Verhalten bezüglich K , jedoch auf Kosten einer höheren Zahl an Iterationen, wie in Abbildung 12 zu sehen.



(a) Zahl Iterationen

(b) Erreichter Wert der Gewinnfunktion

Abbildung 12: Vergleich mit $\Delta t = 4$, $tol = 10^{-6}$, $h_{div} = 4$ und $A \in \{10^{-2}, 10^{-1}, \dots, 10^4\}$

Insgesamt kann ich nicht sagen, dass eine der beiden Implementierungen des FBS-Verfahrens hier bevorzugt werden sollte. Je nach Wahl der Parameter verhält sich das Runge-Kutta- oder das modifizierte-Patankar-Runge-Kutta-Verfahren besser und bisher ist hier kein Muster zu erkennen.

Es sei auch angemerkt, dass diese Vergleiche das Finden einer passenden Schrittgröße h_n nicht beachtet haben, hier wäre ein Vergleich der Laufzeiten beider Varianten vielleicht aufschlussreich.

5 Zusammenfassung und Bewertung

In meinem Praktikum am ITWM habe ich untersucht, wie Patankar-Verfahren bei einer simplen Pandemie-Modellierung genutzt werden könnten. Um dies zu Erforschen, habe ich mit verschiedenen Ansätzen gearbeitet und dabei neue numerische Verfahren kennengelernt. Ich habe täglich mit Mathematikern und auch mit mathematischen Inhalten gearbeitet und bin dabei auf einige interessante Probleme gestoßen, mit denen ich nie allein gelassen wurde. Leider habe ich für meinen zeitintensivsten Ansatz keine eindeutigen Ergebnisse erhalten, doch ich habe trotzdem viel über Forschung mit

Mathematik gelernt.

Zukünftigen Studierenden würde ich das ITWM als Praktikumsstelle sehr empfehlen. Die Arbeit war sehr interessant, ich wurde gut und intensiv betreut und auch in der Stadt Kaiserslautern hat es mir gefallen.

Literatur

- [1] SciPy community. `scipy.integrate.odeint`, 2024. Zuletzt aufgerufen 21 Mai 2025.
- [2] Lawrence C. Evans. Vorlesungsskript zu an introduction to mathematical optimal control theory spring, 2024 version, 2024.
- [3] Stefan Kopecz and Andreas Meister. Vorlesungsskript zu numerik gewöhnlicher differentialgleichungen, Januar 2015.
- [4] Suzanne Lenhart and John T. Workman. *Optimal Control Applied to Biological Models (1st ed.)*. Chapman and Hall/CRC, New York, 2007.
- [5] Qianxiao Li. Vorlesungsskript zu optimal control theory, 2020.
- [6] Jan Mohring, Neele Leithäuser, Jaroslaw Wlazło, et al. Estimating the covid-19 prevalence from wastewater. *sci rep* 14, 14384 (2024). June 2024. PREPRINT (Version 1) available at Research Square [<https://www.nature.com/articles/s41598-024-64864-1>].
- [7] Wikipedia. Explizites euler-verfahren, 2024. Zuletzt aufgerufen 21 Mai 2025.
- [8] Wikipedia. Finite-differenzen-methode, 2024. Zuletzt aufgerufen 23 Mai 2025.
- [9] Wikipedia. Gradientenverfahren, 2024. Zuletzt aufgerufen 24 Mai 2025.
- [10] Wikipedia. Klassisches runge-kutta-verfahren, 2024. Zuletzt aufgerufen 21 Mai 2025.
- [11] Wikipedia. Newtonverfahren, 2024. Zuletzt aufgerufen 24 Mai 2025.